

SEARCH — does MILTANK choose better than MAG

Owens: `engine/miltank.js` , `engine/rollout_leaf.js` , the bring/lead search, the opponent model, the mega choice, post-KO replacement. Design notes in [MILTANK.md](#).

Its one number: the SPRT verdict against the named champion.

May not: fix an engine bug it trips over — file it in [ENGINE.md](#). Patching mechanics mid-run silently invalidates the run, and the run still prints a result.

<!-- GENERATED: engine/status.js -->

```
SEARCH — does MILTANK choose better than MAG
```

```
R1 leaf accuracy    PASS_ON_BASELINE — rollout 67.971% against material's 65.265% on 9,201 positions: +2.
RECORDED, not inferred: n=40, explore=1, key "40@1", stamped by the run that wrote the rows.
```

```
SUPERSEDED 2026-08-04 by this artifact. That sentence was written when only the explore=0 dump existed
```

```
R2 leaf cost        477 boards over 200 games    (2026-08-03 08:22)
```

```
STAMP RECONSTRUCTED, NOT OBSERVED — inferred from commit 05248f23d306; HIGH — written 25s before the c
explore: NOT RECORDED AND NOT PASSED.
```

```
maxTurns: NOT RECORDED AND NOT PASSED.
```

```
games: The artifact's `games` field is the GAMES environment CAP, not a count of games traversed.
```

```
machine: A duration is a fact about a machine under a load.
```

```
R3 divergence       80.2% over 121 decisions (24 agreed, 29 skipped)    (2026-08-04 07:55)
```

```
stamped: n=600@explore=1 (TREE WAS DIRTY — trust source_digests, not the commit)
```

```
R4 does it win      ACCEPT H1 — arm 1 (MILTANK) beats arm 2 (MAG): 55.5% of 535 decisive pairs, 95% CI [5
runs vs engine (newest engine source: engine/medicham2-browser.js 2026-08-04 23:41):
```

```
PRE-CHANGE games.r4-decided.jsonl  2026-08-04 04:41
```

```
PRE-CHANGE games.r4-fixed-part1.jsonl  2026-08-04 02:36
```

```
PRE-CHANGE games.r4.jsonl  2026-08-04 02:33
```

```
PRE-CHANGE games.r4-baseline.jsonl  2026-08-04 01:22
```

```
PRE-CHANGE games.r4-smoke.jsonl  2026-08-04 00:45
```

stamped 2026-08-05 00:26

<!-- /GENERATED -->

R8 — WOBBUFFET re-run, 2026-08-04. VOID. THE TREE MOVED UNDER IT. DO NOT QUOTE THE NUMBERS.

The re-run was authorised ("*rerun wobba*" ... "*yes do the search once engine is all wrapped up*"), was executed at full size, and **produced no usable statement about MAG's exploitability**, because the two things it was measuring both changed while it was measuring them. The old 63.2% is retracted anyway — see below — so the net position is that **MAG's exploitability is now UNMEASURED**, which is a worse place than this session started but a truer one.

What moved, with times, because this is the entire result

what	when	why it is fatal
data/policy-weights.json — MAG itself — was REFITTED	generated: 2026-08-04T22:15:24.522Z	the search loaded the defender at 21:41 and froze it in a temp file; the held-out replay loaded it again at 22:17 , after the refit. The two legs defended with different vectors. New corpus stamp: 8,759 games / 229,339 decisions
engine/board.js written	22:50 → mtime 21:50:36	mid-search, around round 5. Every candidate is scored through <code>dmgMon</code> , so rounds before and after it are not comparable
engine/medicham2-browser.js — the simulator every score goes through	mtime 22:26:57, then four distinct content digests across three sampling windows : <code>0e4b2394edfc</code> (22:29:04) → <code>e9a4215e13d4</code> (22:30:34) → <code>d1a4e497c0e9</code> (22:35:53) → moved again by 22:37:53	sampled with <code>run_stamp.sourceDigests()</code> , content and not mtime. It was still moving forty minutes after the run ended and while this section was being written
the ENGINE census	157/165 when this task was briefed → 164/171 in <code>status.js</code> at 22:31	ENGINE is mid-band, not wrapped

One thing IS stable and it matters for the re-run: `data/policy-weights.json` has held sha256 `5a1930e8926af262` / generated `22:15:24.522Z` since the refit, unchanged across `22:29–22:37`. The defender is settled; the simulator is not.

`data/engine-release.json` **does not exist**. No release has ever been cut, so DIVISIONS rule 1 was unenforceable here in the same way it has always been unenforceable, and `exploit.js` stamps nothing at all — no engine digest, no target digest, no node version. It cannot detect any of the above and did not.

The brief said the engine was wrapped and committed at `96d82cb`. That is not true of the working tree. Reported as observed rather than argued: two content digests ninety seconds apart, printed above. This is not a criticism of the ENGINE band's work — it is the reason the release boundary in `P0.5` exists, and it has now cost a 7,100-game run.

The one thing that IS clean, and it is worth keeping

The mirror control at n=782: 49.7% [46.2, 53.2]. Both legs of the held-out replay ran inside one stable window (`22:17–22:24`: `board.js` stable since `21:50`, weights stable since `22:15`, `medicham2` not touched until `22:26`), so this is a valid measurement of one build. It lands dead on 50, which retires a live worry: the 47.0% and 47.5% round-o mirrors in the two searches are **noise at n=217**, not a seat or pairing asymmetry biasing every other row. `mew.js`'s side alternation is doing its job.

The old 63.2% is retracted regardless, and not because of anything measured today

	2026-07-26	this run (VOID)
features	17	58
games/eval, rounds, seed	220, 18, 90210	220, 24, 90210
mirror control	47.5%, n=217	47.0%, n=217
best challenger vs MAG	63.2% [56.6, 69.3]	~55.8% [49.1, 62.3]~~
held-out replay at unseen seeds	never done	~45.8% [42.3, 49.3], n=782~~

The 63.2% describes a **17-feature** vector on an engine 25 wire-fixes old, computed **before the quality filter existed** — which is exactly why `provenance.js` called it its only `UNSAFE` artifact. It cannot be quoted whether or not a replacement exists, and `docs/MODELS.md` calling it *"the most important number in the repo"* is no longer supportable. **There is now no exploitability number for this project.** That is the honest state.

Two findings that survive the invalidation, because they are about the TOOL

These are properties of `exploit.js` 's search dynamics and of `provenance.js` 's check. Neither depends on which vector was being attacked, so neither is voided by the tree moving.

Finding 1 — THE ATTACK DIED, and a dead search cannot distinguish "safe" from "unsearched"

This is the caveat that matters and it is not the tool's disclaimer, it is a defect in the search:

	2026-07-26	2026-08-04
dimensions searched	17	58
steps ACCEPTED	10 of 18	1 of 24
step scale at the last round ($0.6 \times 0.85^{\text{failures}}$)	0.164	0.0168

`exploit.js` perturbs every coordinate by `gauss() * scale * (|v| + 0.25)` and multiplies `scale` by 0.85 on **every** failure. In 17 dimensions enough steps landed to keep the scale alive. In 58 the step *norm* is $\sqrt{(58/17)} \approx 1.85\times$ larger for the same per-coordinate scale, so round 1 threw the vector off a cliff — **27.7%**, the worst evaluation in either run — and then the geometric decay ran essentially unopposed. From about round 10 onward the challenger was a near-copy of MAG and the "search" was re-measuring the mirror control twenty more times. The 45–50% cluster in rounds 8–24 is that, not evidence.

So even on a still tree this run could not have proved MAG is hard to exploit. A search that takes one step is not a lower bound on anything. The tool's own closing text says the right thing — *"read it as 'this cheap attack failed', nothing more"* — and this time that sentence is doing real work rather than being boilerplate. **Fix the step rule before spending another 7,100 games**, or the re-run on a frozen release will return the same uninformative null for the same reason.

Finding 2 — `provenance.js` CLEARED THIS ARTIFACT AND IT SHOULD NOT HAVE. FILED FOR MEASURE.

Provenance now prints **o UNSAFE** and lists `exploitability.json` as `ok`. That is a false clear, and the mechanism is exact and reproducible:

```
exploit.js read data/policy-weights.json at    21:41    (module load)
data/policy-weights.json was REFITTED at      22:15:24.522
exploitability.json was written at            22:17:57.624
```

The check is `mtime(artifact) < mtime(input)`. The artifact is newer than its input by **153 seconds** and passes — while having been computed from a version of that input which is **34 minutes older**. `provenance.js` is mtime-based and structurally cannot see this; CLAUDE.md already says *"neither can catch an artifact that records a corpus it did not use"*, and this is the sharpest instance of it the project has produced, because the false clear is what *removed the last UNSAFE row*. **The fix is not in provenance.js** — it is that a generator must stamp the **content digest** of every input it read, at the moment it read it, and provenance must compare digests rather than timestamps. `engine/run_stamp.js sourceDigests()` already does exactly this for the leaf sources.

Consequence for anyone reading the gate: `provenance.js --strict` **will now pass, and** `data/exploitability.json` **is still not quotable**. Do not treat the green as the answer.

What `exploit.js` needs before it is re-run. SPECIFIED, NOT APPLIED.

`engine/exploit.js` is not in `docs/DIVISIONS.md` 's ownership table and it produces a claim about whether a number is true, so the fix is proposed here and not made mid-result. Five defects, all observed in this run:

1. **IT STAMPS NOTHING.** No engine digests, no digest of the target vector it read, no node version, no machine, no pool size, no `n_measured / n_unit`. Every other gate in this project carries a stamp and PRIORITIES #20 exists because two of them were missing two fields. This one has none, which is why a mid-run refit of its own defender was invisible. **This is defect zero: fix it first, because it is what would have aborted this run at round 23 instead of after it.**
1. **The step scale is hardcoded** (`let scale = 0.6`) and there is no floor. Expose it, scale the per-coordinate size by `1/√d` so the step *norm* is dimension-invariant, and floor the decay.
2. **Rounds are compared unpaired** — each evaluation uses a different seed (`SEED0 + r*7919`), so a step is accepted on a difference whose standard error is ~4.7 points at n=220. Common random numbers across candidates, exactly as `miltank.js` already does for post-KO replacement, would make a 220-game comparison mean something.
3. **There is no held-out confirmation phase.** The winner should be replayed at fresh seeds and that number, not the selected max, should be the artifact's headline. The scratch generator used here is not in `engine/`, so `provenance.js` does not enumerate `data/exploitability-holdout.json` at all — the confirmation belongs inside `exploit.js` as a `--confirm` phase.
4. **The team pool is not frozen across evaluations.** Each round is a fresh `mew.js` process that rebuilds the pool from the live store, and OPS ingest landed mid-run: `data/games.ladder.jsonl` was written at 22:03:28 UTC, between rounds ~13 and ~15, and the announced pool moved **7,264 → 7,341 distinct clean teams** by the time the held-out replay ran. Both legs of the held-out replay used one snapshot (7,341, announced identically), so *that* comparison is internally clean; the 24-round search is not exactly reproducible from its seed. `MEW_TEAMS` and `engine/mew_farm.js` exist to pin this and were not used.

The corpus and the flags, recorded rather than implied

- **Pool: 7,264 distinct clean teams** at the start of the search, **7,341** at the held-out replay, drawn by `mew.js` through `engine/quality.js loadGames()`. **The quality filter was on** — it is not opt-in, `MEW_TEAMS` was unset, and `--meta-teams` was NOT used, so this is the full clean pool including the Mickey Mouse teams §3 warns about. Clean ladder games available: 7,228 → 7,316.
- Showdown checkout at the pinned commit `20ad99ffc9a5`, announced by every `mew.js` process.
- Defender = `data/policy-weights.json`, shipped: `reweighted_to_closed`. Verified before the run that the top-level `weights` array is **byte-identical** to `weights_reweighted_to_closed` and is the array `magnemite.loadWeights` actually reads, so the defender is the real shipped MAG. That check is not idle: `exploit.js` reads `weights` while `magnemite.js` also reads `weights`, but the file carries three vectors and only one of them ships.
- `--policy score --policy2 score`, both arms MAG's machinery, sides alternated inside `mew.js` by `swapped`. **MILTANK was not involved: this is a measurement of MAG, not of the search.**
- `exploit.js` 's **challenger is arm 2** (`--weights2`), the **OPPOSITE of the SPRT convention** where arm 1 is the challenger. Its seat attribution was re-derived against `mew.js:502-537` before the run and is correct. `tests/test-sprt-arm-sign.js` passes 12/12 and pins the *other* convention — it says nothing about this file.
- Cost: 25 evaluations × 220 games = 5,500 games, ~36 min, one node process at `--conc 6` (the concurrency is hardcoded in `exploit.js`). Held-out: 1,600 games, ~7 min. **~7,100 games and ~45 minutes total. The run is cheap; that is the good news about having to repeat it.** The reason to re-run is the moving tree and the five defects, not the price.

THE RE-RUN. PREPARED, NOT LAUNCHED — and R9 says DO NOT RUN IT IN THIS SHAPE.

*Superseded in part by R9 below. The five defects listed above are now fixed in `engine/exploit.js` and defect 0 is closed. The preconditions and the command below are still the right ones. **But the probe says a 24 x 220 search over 58 weights closes 0.0% ± 0.1 of the distance to a known planted optimum, so running it would buy another uninformative null for another 7,100 games.** Run it only as a deliberately-labelled negative control on the new tooling, or reduce the challenger family to 4–8 numbers first. Read R9 before spending anything.*

Three preconditions, in order, and **each one failed during the 2026-08-04 attempt**:

1. **ENGINE has actually stopped**, verified by content and not by anyone saying so:

```
`` node -e "const RS=require('./engine/run_stamp.js');const a=JSON.stringify(RS.sourceDigests()); setTimeout(()=>{delete require.cache[require.resolve('./engine/run_stamp.js')]; const b=JSON.stringify(require('./engine/run_stamp.js').sourceDigests()); console.log(a===b?'STILL':'MOVING - do not start');},120000)" ``
```
2. **A release is cut on the CORRECTED engine** — the 68 interaction disagreements and the 7 missing mechanics closed — and `exploit.js` is pointed at it. `data/policy-weights.json` is inside the release, so the defender freezes with it and precondition 2's old "record its sha before and after" is now enforced by the tool rather than by a person remembering.

```
`` node engine/engine_release.js cut "post-interaction-matrix engine, for the WOBBUFFET re-run" node engine/engine_release.js list # must read: 0 of 12 files have moved since ` **Cutting is Will's call**, and SEARCH does not cut: the pointer is shared and MEASURE is currently measuring against d3d04b669e18`.
```
3. **The team pool is pinned**: build it once with `engine/mew_farm.js` and export `MEW_TEAMS`, so all 26 evaluations draw the same population and the seed reproduces. `exploit.js` now records the announcement of every evaluation and **writes itself** `void: true` **if the pool moves**, so this precondition is checked rather than assumed — but it still cannot pin it.

Then, one process. **Read R9 first — this is a negative control until the challenger family shrinks.**

```
export SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown
export MEW_TEAMS=$PWD/data/mew-teams-wobbuffet.json # built once by engine/mew_farm.js
node --max-old-space-size=2048 engine/exploit.js \
  --games 220 --rounds 24 --seed 90210 --confirm 800 --conc 6 \
  --release <the id printed by the cut> --tag wobbuffet-e2
```

- **Cost: 25 x 220 = 5,500 games for the search, 2 x 800 = 1,600 for the confirmation. 7,100 games, ~45 minutes, ONE node process at --conc 6**. RAM is the ceiling — check `FreePhysicalMemory` first; at 3.4 GB free this is one process, not two.
- **Output:** `data/exploitability-wobbuffet-e2.json`. `--tag` is deliberate so the run cannot overwrite the void `data/exploitability.json`, which stays void.
- **Read it once, at the end**, and read three fields BEFORE the headline: `search.accepted` (a `search_died: true` makes the rest meaningless), `void` (must be absent), and `pool_announcements` (must hold exactly one line).
- The headline is `headline.kind: "held-out confirmation"`. `searchBest` **is not the result** — see the selection floor in R9, where 25 x 220 returns 56.6% from pure noise.

Then re-verify: `node engine/engine_release.js list` must still read 0 of 12 files have moved , and `node engine/provenance.js` must show `exploitability-wobbuffet-e2.json` verified by CONTENT rather than by mtime. If either fails, the run is void and saying so is cheaper than publishing it.

What is still open, and it is the whole question

Nobody has measured whether MAG is exploitable on the build we ship, and after today nobody has a number at all. The 63.2% is retracted and nothing replaces it: the replacement run is void, and even had it been clean, a one-step search is not a measurement. PRIORITIES.md #18 is **not closed and not merely stale — it is now empty**, with a diagnosis attached. And the caveat MODELS.md already carries still applies: this grades *readability by a prepared opponent*, which is not the same question as *do we win*, and that one has never been measured against a human at all.

R9 — the step rule, FIXED AND PROVED ON A PLANTED OPTIMUM. And the fix is not enough.

Artifact: `data/exploit-step-probe.json` , written by `engine/exploit_step_probe.js` , stamped with `source_digests` from engine release `d3d04b669e18` . R8 said "fix the step rule before spending another 7,100 games". The step rule is fixed. **The probe then says the step rule was never the binding constraint, and the re-run as specified in R8 should NOT be run.**

The verdict in one line

At 58 features and 24 x 220 games the search closes 0.0% ± 0.1 of the distance to a KNOWN planted optimum. It is not a search. No step rule changes that, because the thing that is broken is the ratio between what one step is worth and what 220 games can see.

The number that ends the argument

On the planted objective with a noiseless oracle, **one accepted step at d=58 moves the true win rate by 0.21 points.** Against that:

what the evaluation can resolve	at 220 games
independent seeds per round — what the void run actually did	4.77 pt — 23x larger than the step
perfect common random numbers (the best CRN can ever buy)	0.45 pt — 2x larger than the step

A hill climb cannot accept a step it cannot measure. Everything else below follows from this row.

What the fix is, and what it bought — acceptance rate before and after

`engine/exploit.js` now exports `createClimber()` , and `exploit_step_probe.js` drives **that function**, not a re-typed copy. Four changes:

1. **The proposal is divided by √d**, so $E[||step||^2] = scale^2 \cdot mean(|v_k| + 0.25)^2$ is independent of the number of features. `scale` now means the same thing at 17 and at 58.
2. **Acceptance-targeted multiplicative adaptation** replaces decay-on-failure. `accepted` → `scale × exp(g(1-p*))` , `rejected` → `scale × exp(-g.p*)` , so $E[\Delta \log scale] = g(\hat{p} - p^*)$ and the scale has a **fixed point at the target acceptance rate**. The old rule multiplied by 0.85 on failure and by 1 on success: its only equilibrium was zero, which is where it went.
3. **A stall restart**, which acceptance-targeting alone does NOT give you and which the probe found rather than the diagnosis predicting. Adaptation assumes acceptance falls monotonically as the step grows. Against a *measured* objective that is false at the small end — once a step's true gain drops below the measurement's resolution it becomes invisible and acceptance collapses **as the step shrinks**. Acceptance is non-monotone, near zero at both ends, and a shrink-on-failure rule parks on the wrong one. Measured

before the restart was added: 1% acceptance and 0.1% of the distance closed, **worse than the rule it replaced.**

4. **The acceptance target is 0.05, not the textbook 0.2–0.4,** and that is measured. The classical band is derived for an exact oracle. Swept at $d=58$, 200 x 1200, perfect CRN:

target acceptance	0.02	0.05	0.10	0.25	0.40
distance closed	19.4%	19.9%	15.9%	6.7%	3.6%

The textbook band is the worst end of the sweep. Steps must be big enough to be **seen**, not merely big enough to be good.

Acceptance rate, $d=58$, 24 rounds, 40 independent runs per arm:

noise model	rule	accepted	distance closed
noiseless oracle	legacy	8.7/24 (36%)	6.3% $\pm$ 0.5
noiseless oracle	fixed	7.4/24 (31%)	6.1% $\pm$ 0.4 ($z = -0.4$, no difference)
independent seeds — as the void run ran	legacy	2.4/24 (10%)	-1.5% $\pm$ 0.5
independent seeds	fixed	2.5/24 (10%)	0.0% $\pm$ 0.1 ($z = 3.1$, fixed better)
perfect CRN	legacy	0.8/24 (4%)	1.4% $\pm$ 0.3
perfect CRN	fixed	0.5/24 (2%)	0.2% $\pm$ 0.1 ($z = -3.7$, legacy better)

Read honestly, three things, and the third is the uncomfortable one:

- **The 1-of-24 acceptance in the real run is reproduced** — the toy gets 2.4/24 at the same dimension, same games, same noise model, without being tuned to.
- **The fix's only measured win is that it stops the search moving BACKWARDS.** Under the noise the run actually had, the legacy rule closed **-1.5%**: it accepted upward noise flukes and ratcheted away from the optimum. That is what "1 of 24" was doing.
- **Under CRN the legacy rule is BETTER at this budget**, and the reason is instructive rather than embarrassing: its un-normalised step is $\sqrt{58} = 7.6x$ larger in norm, which is the right direction when the measurement is coarse. That is what moved the acceptance target to 0.05. At 5,280 games both numbers are ~ 0 against a 25-point edge (0.05 vs 0.35 win-rate points), so this is a comparison of two zeroes and neither rule is worth running at that budget.

The two walls, and neither is the step rule

Wall 1 — evaluations. A (1+1) climb makes progress at $\sim 1/d$ per evaluation. 24 evaluations in 58 dimensions is 0.4 of one such unit, and the probe confirms it: **with a NOISELESS oracle the ceiling at 24 rounds is 6.1% of the distance.** At $d=17$ the same 24 rounds close 19.9%. The 2026-07-26 run was not luckier, it was in a smaller space.

Wall 2 — resolution. The table at the top. Below it, no number of rounds helps: the independent -noise column of the budget sweep is flat at 0.0% from 5,280 games to **960,000**.

rounds x games	total	independent	perfect CRN
24 x 220	5,280	0.0% $\pm$ 0.1	0.2% $\pm$ 0.1
100 x 220	22,000	-0.0% $\pm$ 0.1	1.0% $\pm$ 0.5
200 x 220	44,000	-0.0% $\pm$ 0.1	1.2% $\pm$ 0.7
200 x 1200	240,000	-0.0% $\pm$ 0.3	19.9% $\pm$ 2.1
400 x 1200	480,000	-0.0% $\pm$ 0.3	29.2% $\pm$ 2.8
800 x 1200	960,000	-0.0% $\pm$ 0.3	36.8% $\pm$ 3.4

Cheapest split that closes a material (>25%) fraction of the distance: 960,000 games, and only if common random numbers couple perfectly. That is not a run this project should schedule, and it is the honest reason to stop rather than a reason to argue for a bigger machine.

The lever that IS affordable: search fewer numbers

The other end of the same trade. At the affordable $24 \times 220 = 5,280$ games, with CRN, with the optimum planted **inside** the searched space:

dimensions searched	4	8	17	30	58
distance closed	49.7% ± 2.2	30.7% ± 3.0	5.9% ± 1.3	1.2% ± 0.2	0.2% ± 0.1

A 5,280-game search buys a real answer about a family of roughly 4 to 8 numbers and nothing at all about a family of 58. So the re-run's design question is no longer "how do we step" — it is **what low-dimensional reparameterisation of MAG's policy is worth attacking**: feature groups, a handful of scalars, a temperature. That is a SEARCH design item and it is now the blocker on R8, ahead of the engine.

Explicitly NOT tested, so it is not smuggled in: perturbing only k of the 58 raw coordinates. On a *dense* optimum a random k -subspace caps at $1 - \sqrt{1 - k/d}$ of the distance no matter the budget, and nobody has measured whether the real exploit direction is sparse. The table above plants the optimum inside the searched space, which is the right question for a reparameterisation and the wrong one for a sparse mask.

The selection floor — what the OLD headline reported when nothing was found

Pure arithmetic on the binomial, and nobody has ever printed it beside the headline. Under the null that every candidate is exactly as good as MAG, the **maximum over R+1 evaluations** is still:

evaluations x games	mean reported "best"	95th pct	99th pct
19 x 220 (the 2026-07-26 run)	56.2%	59.5%	60.9%
25 x 220 (the void run)	56.6%	59.5%	61.4%
25 x 800	53.5%	55.1%	56.0%
201 x 1200	54.0%	54.9%	55.6%

The void run's search-best of 55.8% is BELOW the floor its own procedure produces from pure noise. It was never a finding. The retracted 63.2% sits above the 99th percentile of its floor, so *that* one is not explained by selection alone — which changes nothing about its retraction, since the objection to it was provenance and a 17-feature vector on a 25-wire-fix-old engine, not selection.

This is why the artifact's headline is now the held-out `--confirm` leg and why `searchBest` carries the literal label `SELECTION-BIASED`, not the headline.

What `exploit.js` now does. IMPLEMENTED, and defect zero is closed.

- **Opens** `engine_release.open()` **and REFUSES TO RUN without one.** Prints `REFUSING TO RUN: no engine release has been cut`.
- **Reads the DEFENDER out of the snapshot** — `REL.path('data/policy-weights.json')` — so the file that moved on 2026-08-04 cannot move again. Verified: the refusal path fires today, naming `engine/medicham2-browser.js`, `engine/tags.js`, `data/tags.json`.
- **Re-checks** `drift()` **after every single evaluation** and aborts mid-run with `void: true` and the file list. This is the guard that would have stopped the void run at round 23 instead of after it.
- **Stamps** `REL.stamp()` — 12 content digests, the release id, the Showdown commit — plus the target vector's own sha12, node version, machine, `--conc`, `n_measured / n_unit`, and every `MEW: N distinct clean teams` announcement it saw. **If the pool moved between evaluations the artifact writes itself** `void: true`.
- **Common random numbers** across all search rounds (`--no-crn` restores the old per-round seeds).

- A `--confirm` **phase** at seeds the search never touched, plus a fresh mirror control, and THAT is the headline. It used to live as a scratch file outside `engine/` , which is why `provenance.js` never enumerated `data/exploitability-holdout.json` .
- **Says out loud when the search died:** `THE SEARCH DID NOT MOVE...` it is evidence this search did not look , and `search_died: true` in the artifact.
- `--legacy-step` reproduces the 2026-08-04 rule exactly, so the comparison above stays runnable.

`data/exploitability.json` and `data/exploitability-holdout.json` are **left void and were not regenerated**.

FILED FOR MEASURE — the frozen release is not a loadable engine, and its freeze list has holes

Tripped over while wiring `exploit.js` ; **not fixed here, because changing `SOURCES` changes every release id and two divisions are measuring against `d3d04b669e18` right now.**

`REL.require()` — the usage `CLAUDE.md` documents — throws for **4 of the 12 frozen sources**:

```
OK    engine/medicham2-browser.js      FAIL  engine/board.js      -> Cannot find module './mc_key.js
OK    engine/tags.js                  FAIL  engine/rollout_leaf.js -> Cannot find module './mc_key.js
                                           FAIL  engine/position_features.js -> Cannot find module './mc_key
                                           FAIL  engine/champions_sim.js  -> Cannot find module './showdown_
```

The example in `CLAUDE.md` happens to be the one file that works. **Six files are reachable from the freeze list and are not in it**, and they are not inert:

unfrozen	required by	why it can change a number
<code>engine/mc_key.js</code> (16 KB)	<code>board.js</code> , <code>position_features.js</code> , <code>rollout_leaf.js</code>	decides which dex row a species resolves to
<code>engine/lookup.js</code> (5 KB)	<code>board.js</code> , <code>mc_key.js</code>	the lookup path underneath it
<code>engine/set_priors.js</code> (40 KB)	<code>champions_sim.js</code>	what an unknown set is filled with — every self-play game
<code>engine/smogon_priors.js</code> (17 KB)	<code>set_priors.js</code>	same
<code>engine/quality.js</code> (15 KB)	<code>set_priors.js</code>	decides the team pool
<code>engine/showdown_path.js</code> (4 KB)	<code>champions_sim.js</code>	which Showdown checkout is loaded

So the release is a valid **digest set** and not yet a loadable **engine**, and its claim to freeze "every file whose content can change a number" is currently false for six files. `exploit.js` works around it by reading `FEATURES` from the live `board.js` *after* proving zero drift, which is equivalent and verified — and which is exactly the kind of workaround that stops being safe the day someone passes `--allow-drift` . It says so in a comment at the line.

FILED FOR MEASURE — `provenance.js` marks every RELEASE-BASED measurement UNSAFE

Found by being the first artifact to stamp `source_digests` from a release. `provenance.js:650` resolves a stamped input against the **live tree** (`digestOf(src)`). A measurement that reads a frozen release is, by design, computed from bytes the live tree has moved past — so the check that was built to catch a moving tree now fires on the artifacts that handled a moving tree correctly:

```
exploit-step-probe.json  UNSAFE  COMPUTED FROM DIFFERENT CONTENT —  
data/policy-weights.json was 5a1930e8926a at read time, is 01bc43936324 r
```

That digest moved because MEASURE's refit landed, which is exactly the event the release exists to survive. The artifact is **correct**; it names release `d3d04b669e18` and read that release's bytes.

Suggested shape of the fix, which is MEASURE's to make: when an artifact carries `engine_release`, verify its `source_digests` against **that release's manifest** and report `ON RELEASE <id>` — with a separate, non-UNSAFE line saying how far the live tree has since drifted. `UNSAFE` should remain for an artifact whose stamped digest matches neither the live tree nor any release, which is the case that means "computed from bytes nobody can name".

Until that lands, the count printed as `0` verified by CONTENT digest, `92` by mtime alone **cannot reach 1 for any release-based artifact while ENGINE or MEASURE is working** — the row leaves the mtime-only list and lands in `mismatch` instead of `verified`. The ratchet in `data/provenance-stamp.json` still falls (that list is what is ratcheted), so this does not block the ratchet; it does mean the headline verified-count understates the fix.

A second, larger limitation, stated because it is structural rather than a bug: `exploit.js` spawns `mew.js`, which loads the LIVE engine. No release can prevent that without the child being runnable from the snapshot. Detection (drift-check every evaluation, abort, self-declared void) is what is implemented; prevention is not, and pretending otherwise would be the more dangerous choice.

R10 — the reparameterisation memo. ANALYSIS, 2026-08-05. WILL DECIDES; nothing here was run against MAG.

R9 ended with "search fewer numbers" and left open WHICH numbers. This section brings the concrete options. Every figure traces to an artifact; the two new ones are `data/exploit-step-probe-reparam.json` (the R9 toy swept over family sizes 4–12 at the real budget and at twice it, written by `engine/exploit_step_probe.js --reparam` — same `runOne`, same `createClimber`, no games) and the fitted vector itself, `data/policy-weights.json` (generated `2026-08-04T23:37:26.954Z`, corpus 8,856 games / 231,722 decisions, frozen in release `6e43710396db` as `01bc43936324`).

The arithmetic that frames every option (data/exploit-step-probe.json)

- One accepted step at $d=58$ moves true win rate by **0.202 pt**; 220 games resolve **4.77 pt** (independent seeds) / **0.45 pt** (perfect CRN). The step is invisible, so the search cannot climb.
- Largest family the affordable $24 \times 220 = 5,280$ -game budget can actually search: **about 4** numbers (`largest_searchable_family_at_5280_games`).
- The toy plants a 25-pt edge ($p_{\text{Max}} 0.75$), so "distance closed" reads as "fraction of the family's available edge captured". The confirm leg (`--confirm 800`) certifies nothing smaller than **~3.5 pt** ($1.96 \cdot 50 / \sqrt{800}$), whatever the search finds.

Family sizes at the real budget and at 2x, measured, not extrapolated (`data/exploit-step-probe-reparam.json` ; fixed rule; distance closed $\pm$ SE over 40 runs; the truth about CRN coupling in real games is UNMEASURED, so both brackets are printed):

family size	24 x 220 crn	24 x 220 indep	48 x 220 crn	48 x 220 indep	24 x 440 crn
4	49.9 $\pm$ 2.2%	37.0 $\pm$ 3.9%	63.9 $\pm$ 2.4%	49.3 $\pm$ 3.7%	51.6 $\pm$ 2.7%
6	42.3 $\pm$ 2.8%	18.6 $\pm$ 3.3%	52.6 $\pm$ 2.6%	26.9 $\pm$ 3.8%	45.6 $\pm$ 2.2%
8	30.2 $\pm$ 2.9%	11.3 $\pm$ 2.5%	42.8 $\pm$ 3.5%	15.8 $\pm$ 3.2%	36.5 $\pm$ 2.5%

family size	24 x 220 crn	24 x 220 indep	48 x 220 crn	48 x 220 indep	24 x 440 crn
12	21.1 ± 2.4%	1.1 ± 0.6%	31.7 ± 3.1%	4.8 ± 1.6%	26.5 ± 2.3%

Three design facts fall out before any family is chosen: **doubling ROUNDS beats doubling games-per-round in every crn row** (the climb is rounds-starved, exactly as R9's O(d)-evaluations argument says); **d=4 is the only size that stays searchable in the independent bracket**, i.e. the only one whose verdict does not depend on how well CRN couples in real battles; and a doubled budget (48 x 220 + 2 x 800 confirm ≈ 12,160 games, ~75 min by R8's timing) buys d=4 nearly two-thirds of its family edge.

One structural note before the families

MAG **samples** its softmax rather than taking the argmax (`engine/magnemite.js:12-17`), so a global temperature is a real, playable lever — `c·w` and `w` are different players here, unlike in an argmax policy. That is why F1 below is allowed to spend a parameter on it.

The families

F1 — AXIS-4: temperature, prior, kill, initiative. RECOMMENDED. `w'_k = exp(τ) · exp(a_P·[k=priorLogP] + a_K·[k ∈ KILL] + a_I·[k ∈ INIT]) · w_k`, searched over `z = (τ, a_P, a_K, a_I)` from `z0 = 0` (so the incumbent MAG is the start point by construction). KILL = {koTarget, dmgFrac, tgtMayProtect, killsIsRoll, killsThreat, koFirst, protectThreatened}; INIT = {movesFirst, priority, speedSwing, diesBeforeMoving} — the blocks as `board.js` FEATURES declares them (release digest `54e3d2ca9f85`). *Why these four axes, from the fit itself:* `priorLogP` is the single most-determined coordinate in the whole vector (`w` +0.1474, SE 0.0026 — the fit pins it hard **for resemblance**, and resemblance is exactly the objective that cannot certify it **for winning**); the kill and initiative blocks are where the fit is weakest — `koTarget` +0.0348 ± 0.0170, `killsThreat` **-0.0610** ± 0.0131 (killing the thing about to kill you fitted *negative*), `priority` -0.0053 ± 0.0159 and `movesFirst` +0.0075 ± 0.0126 (both indistinguishable from zero). A challenger that wants to beat MAG by wanting kills and initiative more than people do lives exactly here. *Can express:* greedy↔noisy play, prior-reliance up or down, uniform kill-hunger, uniform initiative-hunger, and their combinations. *Cannot:* rotate within a block (raise `koTarget` while lowering `dmgFrac`), touch the switch/support/dead-move axes, flip any individual sign, or form any interaction the 58 features do not already carry. *Resolution:* 49.9 ± 2.2% of the family edge at 5,280 games (crn) and **37.0 ± 3.9% even at the independent bracket**; 63.9 ± 2.4% at the doubled budget. A *WOBBUFFET* null here proves: no re-mix of sharpness/prior/kill/initiative beats shipped MAG by more than the ~3.5-pt confirm floor. It says **nothing** about within-block, switch-axis, or novel-interaction exploits, and MAG's general exploitability stays unmeasured. A *positive* hands MEASURE a named, four-number direction to test as a refit objective.

F2 — BLOCK-8: one log-gain per board.js feature family. The eight blocks as the FEATURES list groups them: targeting/move-quality (13), dead-moves (9), order (4), kill (10), disruption/stages (9), switch (8), support/value (4), prior (1) — 58 accounted for. *Can express:* everything F1 can, plus the switch, support, dead-move-discipline and disruption axes. *Cannot:* within-block rotation or sign flips, same as F1. *Resolution:* 30.2 ± 2.9% at budget (crn) but **11.3 ± 2.5% at the independent bracket** — its verdict leans on CRN coupling nobody has measured; 42.8 ± 3.5% at 2x. A *null* proves: no block-level retuning of MAG's vocabulary beats it at the floor. Broader statement than F1's, bought with a real risk that the search under-resolves and the null is about the noise, not the family.

F3 — FLAT-6: the six flattest Fisher directions of the fit. BLOCKED ON MEASURE.

`fit_policy.js standardErrors()` already computes the full observed information `H` (`:663-700`) and publishes only the diagonal of `H-1`. The 4–8 bottom eigenvectors of `H` (preconditioned) are the directions the resemblance likelihood constrains LEAST — the largest moves a challenger can make per unit of "still plays like the corpus". Dense directions, not an axis mask, so the probe's planted-inside-the-family table applies (d=6: 42.3 ± 2.8% at budget crn, 52.6 ± 2.6% at 2x). *Cannot:* move in stiff directions — which is precisely

where a deliberately non-human exploit would live, so this family is biased toward subtle exploits and blind to flagrant ones. Also unstable across refits (eigenvectors rotate with the corpus), so the H snapshot must be pinned in the artifact. *Blocked*: needs MEASURE to export H or its eigendecomposition; filed as a one-flag change to `fit_policy.js`, not made here — MEASURE's file.

F4 — SPARSE-8: the eight raw coordinates the fit barely pins ({priority, movesFirst, switchKOFast, tgtHurt, switchKOSlow, pivots, koTarget, allyHit} — the top of the SE/|w| ranking). Interpretable and directly refittable, **but it is an axis-aligned subspace of the raw space**, and the probe's own warning applies verbatim: against a dense exploit direction it caps at $1 - \sqrt{1 - 8/58} = 7.2\%$ of the distance at any budget. Its null is therefore the weakest of the four. Fit only as a cheap confirmatory second arm if F1 finds something, or not at all.

Implementation cost, so the decision is priced

`exploit.js` climbs in z-space with `x0 = 0_k` and maps `z → w(z)` before `writeweights` — one `--family` flag, ~30 lines, in the file SEARCH already maintains; the climber, CRN, drift-abort, pool-void and confirm phases all apply unchanged. The artifact must stamp the family definition (the block memberships and the mapping) beside the release id, or the result names a challenger nobody can rebuild.

Recommendation, marked

Run F1 at the doubled budget: 48 x 220 search + 2 x 800 confirm ≈ 12,160 games, one process, after the current sweep finishes. It is the only family whose resolution survives the independent bracket, its four axes are the four questions this project keeps asking about MAG in prose (too timid? too human? too slow to take kills? too willing to lose initiative?), and either outcome is actionable: a null retires the block-gain hypothesis at a stated floor, a positive is a refit direction with names on it. F2 second if F1 nulls and Will wants the switch/support axes covered. F3 waits on MEASURE. F4 only as a confirmatory arm. And the standing caveat carries: every family grades *readability by a prepared opponent under our own leaf*, which is not "do we win", and the leaf's calibration is MEASURE's open item — a null can be about the leaf, not the search.

What the 2026-08-04 mega-weather fix invalidates

Stated plainly rather than left to be inferred, because a leaf change that moves values and is not declared is how a stale number survives. **Everything below was computed on a leaf in which a mega never brought its weather.** None of it is wrong about its own arm; all of it describes a build that no longer exists.

	why it is affected	what it needs		
R1 leaf accuracy, and the explore sweep	every position scored through <code>rolloutWinProb</code> ; 6.4% of corpus boards carry a mega setter in clear weather and move by ~9.7 pt	re-run at the release boundary. The <i>sign</i> is very unlikely to move; the point estimate will		
R2 leaf cost	already under re-run (PRIORITIES #14). Cost measured unchanged here (17.53 → 16.93 ms at n=40), so this fix is not the reason	nothing extra		
R3 divergence	<code>rolloutAfterActions</code> moved on the same boards, by more (mean \	$\Delta \backslash$	18.3 pt at n=24)	re-run
R4 the SPRT	every leaf call in both arms	re-run. Note both arms shared the defect, so it partly cancels — which is precisely the failure mode CLAUDE.md names, and is not a reason to keep the number		
the leaf calibration	already void for the preview under PRIORITIES #38; the in-game half is now void too	MEASURE, after the boundary		

	why it is affected	what it needs
(53.22% / 50.99%)		

This is a release-boundary matter, not a footnote — PRIORITIES Po.5. The runs on disk were already PRE-CHANGE against the engine; they are now PRE-CHANGE against the leaf as well, and the leaf is SEARCH's own file. Do not start a wide run until the boundary is cut.

Po.5 — THE FROZEN ENGINE RELEASE. DESIGNED AND PREPARED 2026-08-04. NOT CUT.

docs/DIVISIONS.md rule 1 says SEARCH plays a frozen, named engine release and never HEAD. **There is no such release and there never has been**, so the rule has been a sentence rather than a mechanism, and every SEARCH baseline on disk is attributed by status.js comparing mtimes — the one thing run_stamp.js says in its own comments is not evidence, because a checkout moves an mtime without moving code.

This section is the mechanism, the freeze list, the re-run order and the commands. **Cutting it is Will's call**, because the cut triggers the refit and seven restamps.

1. What identifies a release: a DIGEST SET in a file, not a tag

data/engine-release.json , written by a cut and never hand-edited:

```
{
  "release": "E1-2026-08-05",
  "cut": "2026-08-05T00:00:00.000Z",
  "supersedes": null,
  "commit": "<sha>",
  "dirty": false,
  "digests": {
    "engine/rollout_leaf.js": "02c57b55e929",
    "engine/medicham2-browser.js": "7649d0760a88",
    "engine/board.js": "bcf2dab9dc6f",
    "data/engine-data.js": "96a94b7fADF7",
    "data/abra-tags.js": "ea5b89c2afcd"
  },
  "claims": { "census_live": 115, "differential": "1/400 @ seed 20260804", "tests": "run-all green" }
}
```

Why a digest set and not a git tag. A tag names a commit, and this repo has already published a result whose own stamp reads *"TREE WAS DIRTY — trust source_digests, not the commit"* (R3, in the generated block above). An unattended auto-commit publishes on a timer here, so a commit id is not a stable statement about what a process loaded; the bytes are. A tag is still worth pushing as a human handle (git tag engine/E1-2026-08-05), and it is a **convenience, not the authority**.

The digests come from engine/run_stamp.js — sourceDigests() **and** gitState() — **not from a new hasher**. That is not tidiness: miltank.js was hashing **4 files with sha1** while run_stamp.js hashes **5 with sha256**, so data/abra-tags.js — the file ENGINE rewrites most — was invisible to a MILTANK stamp and visible to every other gate's. Two definitions of "the engine these numbers describe" is the choiceLock failure in a new costume. miltank.js buildStamp() now calls RS.sourceDigests() and the divergence is closed.

2. What exactly is frozen, and why each file

The release freezes **every file whose bytes can change a rollout's value**. That is `run_stamp.LEAF_SOURCES` , which already exists and already has the right membership:

file	why it is in
<code>engine/medicham2-browser.js</code>	the simulator. Damage, priority, abilities, the playout loop
<code>engine/rollout_leaf.js</code>	the playout and the field boundary. <code>applyField</code> alone has moved two published numbers this week
<code>engine/board.js</code>	<code>dmgMon</code> builds every rollout body; <code>candidates()</code> is the menu
<code>data/engine-data.js</code>	stats, moves, items — the table every body is built from
<code>data/abra-tags.js</code>	every mechanic param the engine reads. The census lives here

Deliberately OUT, each for a stated reason:

- `engine/miltank.js` **is NOT frozen**. It is the thing under test. Freezing the player inside the release would make an H2H between two players impossible to name.
- `data/policy-weights.json` **is NOT frozen**. MAG's fit is a different invalidation edge with a different authority, and conflating them is why this needed writing down.
- `engine/mew.js` , `engine/sprt.js` **are NOT frozen**. DIVISIONS: MEASURE's tools sit beside the graph and invalidate nobody.

ENGINE's freeze list is not the feature path, and the two authorities must not be swapped.

question	authority	what it answers
is the fit stale?	<code>node engine/feature_fixture.js --check data/policy-weights.json</code>	do the fitted weights still mean what <code>board.js</code> computes. Ran clean 2026-08-04: <i>"agrees with board.js on every fixture board"</i>
is a rollout comparable?	<code>data/engine-release.json</code> digests	did the simulator move under the run

They overlap on `board.js` only, and they can disagree in both directions: the fit is currently **clean** while every rollout on disk is **not comparable**. Reporting one as the other is the silent-default failure DIVISIONS names as the cost of a boundary.

3. How a run declares its release, and how `status.js` marks `PRE-RELEASE`

Implemented and proved, in SEARCH's own file. `miltank.js buildStamp()` hashes its own worktree at module load — not at the first row, so it cannot describe a file edited underneath a running process — and resolves three states against `data/engine-release.json` :

stamp reads	meaning
<code>release: "E1-...", release_status: "ON_RELEASE"</code>	every frozen file hashes to the release
<code>release: "E1-...", release_status: "PRE-RELEASE" , release_moved: [files]</code>	the run is not on the release, and it names which files differ
<code>release: "UNRELEASED"</code>	no cut has ever happened — what every stamp reads today

Both non-trivial branches were exercised rather than assumed: with a synthetic release file the resolver returned `ON_RELEASE` , and with one digest perturbed it returned `PRE-RELEASE release_moved: ["data/abra-tags.js"]` .

The change `status.js` needs — SPECIFIED, NOT APPLIED, because `status.js` is MEASURE's file. Today `status.js:315-331` finds the newest engine-source **mtime** and prints `PRE-CHANGE` for any run file older than it. Replace the comparison, keep the line:

1. read `data/engine-release.json` ; if absent, print `NO RELEASE CUT – rule 1 of DIVISIONS.md` is unenforced and fall back to today's mtime inference **labelled as an inference**;
2. for each run, read its stamp (`*.meta.json` sidecar for a gate artifact, the `_stamp` row for a `MILTANK_TIMING` shard, `_stamp` in the games jsonl for a mew run) and compare its `engine_digests` to the release's `digests` ;
3. print one of `ON <release>` , `PRE-RELEASE (<files that moved>)` , or `NO STAMP` .

`NO STAMP` **must be its own state and must not read as current.** An unstamped run is not evidence about any build, which is strictly worse than a run known to be old — that is the whole finding behind `run_stamp.js` existing.

The resolver in `miltank.js` is written to be **lifted into** `engine/engine_release.js` when MEASURE lands the `status.js` half, so `status.js` , `run_stamp.js` and `miltank.js` share one implementation. **Do not write a second one.**

4. What must be re-run once the cut lands, ordered by cost

Cost is stated in leaf calls and payouts, not minutes, because this file's own R2/R6 sections say a duration is a fact about a machine under a load and R2 is being re-run for exactly that.

order	gate	unit of work	shares a corpus with	why it must be re-run		
1	R2 leaf cost	~477 leaf calls	R1 (same walker, same stride)	already owed (PRIORITIES #14); the weather and terrain fixes make payouts longer, so every cost figure quoted downstream is a lower bound until this lands. Run it first — every other estimate below is priced off it		
2	R1 + the explore sweep	9,201 positions × 3 explore arms ≈ 27.6k leaf calls ≈ 1.1M payouts at n=40	R2, R3, R5	every position scored through <code>rolloutWinProb</code> . The sign is very unlikely to move; the point estimate will		
3	R3 divergence	121 decisions at n=600, two searches plus the self-disagreement control	R1, R5	<code>rolloutAfterActions</code> moved by mean \	Δ\	18.3 pt on the same boards
4	R7 timing distribution	12 self-play games	R6a (identical command plus <code>MILTANK_CLOCK=1</code>)	R6's figures describe a build that no longer exists and are lower bounds		
5	the in-game leaf calibration	MEASURE's, corpus-sized	R1, R3, R5	measured on a leaf that could not read weather. MEASURE's item, not SEARCH's		
6	R5 action-ranking backtest	~2,000 decisions × ~63 cells × n=200 ≈ 25M payouts, 4 shards	R1, R3	never run; it is the measurement that decides whether the leaf is worth its cost		
7	R4 the SPRT	~420 decisive pairs, self-play	R6c only	every leaf call in both arms. Last, because 1–6 can redirect it		

What genuinely shares a corpus. R1, R2, R3 and R5 all walk the same clean open-sheet games through `joint_rows.build`'s `onBoard` observer with a stride, so they sample from one population and should record **the same corpus id and the same stride** — and R3 is computable as a **by-product of the R5 pass**,

because R5 already enumerates the menu at each decision point that R3 compares two searches over. R4, R6a, R6c and R7 are self-play through `mew.js` and share nothing with the corpus gates; R7 and R6a are the same 12-game command differing by one environment variable.

What does NOT need re-running: the live-budget derivation (read out of the Showdown source, not measured on our engine), the request-length distribution over 30,396 ladder games (a property of the store), and R6b's forfeit answer, which is arithmetic on those two.

5. THE TRAP THAT WOULD MAKE ALL OF THIS A NULL — instrumented 2026-08-04

PRIORITIES Ob: `--miltank` **with** `--policy random` **searches nothing and looks completely normal.** Every `chooseMove` bails silently, and an H2H arm can therefore run a whole job having never called the leaf while still printing a win rate.

Reproduced, and it is worse than filed. A 2-game run with `--policy random --miltank` finished clean, printed `MEW done: 2 games (0 discarded)`, and wrote **no** `MILTANK_TIMING` **file at all** — not an empty one, none — because every decision bails before the recorder is reached. The `--reduce` step answered that with an ENOENT stack trace, which reads like a broken tool rather than a run that never searched.

So the counter now exists and every re-run must read it before it reads anything else.

`engine/miltank.js` counts leaf entries and playouts, stamps the running total on every timing row, and `--reduce` publishes:

```
"search": { "leaf_calls": 1142, "playouts": 28415,
            "decisions_with_zero_leaf_calls": 0, "zero_leaf_pct": 0, "VERDICT": "ok" }
```

with `VERDICT` reading `THE SEARCH NEVER RAN` — this artifact is not a measurement of `MILTANK` at zero, and `--reduce` on a missing file returning a named verdict instead of a stack trace. Verified on a 3-game smoke: 36 decisions, 1,142 leaf calls, 28,415 playouts, 0 decisions with zero leaf calls.

The rule for every command below: `MILTANK_TIMING` **is set on BOTH arms, and no verdict is read until** `search.leaf_calls > 0` **on both.** For the corpus gates (R1/R3/R5), which do not go through `miltank.js`, the equivalent guard is the generator's own `nulls / skipped` counters — a gate whose rows are mostly nulls is the same failure wearing the other hat.

6. The commands. PREPARED, NOT RUN.

`SHOWDOWN_PATH` is required by all of them. Check `FreePhysicalMemory` before choosing a process count — it was **3.4 GB** when this was written, which is one to two processes, not six.

Step 0 — the cut itself. Will's call, and it triggers the refit and the restamps.

```
# preconditions, all three, in this order
git status --porcelain          # must be clean, and no rebase in progress
node tests/run-all.js          # the census must not be down
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
  node engine/feature_fixture.js --check data/policy-weights.json

# write data/engine-release.json from run_stamp's digests – this is the cut
node -e "const RS=require('./engine/run_stamp.js'),cp=require('child_process'),fs=require('fs');
const d=RS.sourceDigests(); delete d.note;
const commit=cp.execSync('git rev-parse HEAD').toString().trim();
const dirty=cp.execSync('git status --porcelain -- '+Object.keys(d).join(' ')).toString().trim();
if(dirty) throw new Error('REFUSING TO CUT A DIRTY TREE:\n'+dirty);
fs.writeFileSync('data/engine-release.json', JSON.stringify({
  release:'E1-'+new Date().toISOString().slice(0,10), cut:new Date().toISOString(),
  supersedes:null, commit, dirty:false, digests:d}, null, 2)+'\n');
console.log('cut', require('./data/engine-release.json').release);"

git tag engine/E1-$(date +%F)    # the human handle, NOT the authority
node engine/status.js --write
```

The dirty-tree refusal is a feature, and it fires today. Dry-run 2026-08-04 21:01 UTC: the guard refused, and in the eight minutes either side of it `engine/medicham2-browser.js` went `7649d0760a88` → `3653b857dc29`, `engine/board.js` went `bcf2dab9dc6f` → `88506029c850` and `data/abra-tags.js` went `ea5b89c2afcd` → `facd3f2f50b4`. **Three of the five frozen files moved while this section was being written**, which is the entire argument for the boundary, observed rather than asserted. The cut happens after ENGINE lands and commits — the answer is never to drop the guard.

Step 1 — R2, first, because everything else is priced off it:

```
SHOWDOWN_PATH=... GAMES=120 EVERY=3 N_LIST=10,40,200 EXPLORE=1.0 MAXTURNS=60 \
  node --max-old-space-size=4096 engine/rollout_r2.js
```

`EXPLORE` and `MAXTURNS` are passed **explicitly** — the committed artifact was measured at the defaults (explore 0, maxTurns 20) while the shipped leaf runs 1.0 and 60, and not passing them is the whole of PRIORITIES #14.

Step 2 — R1 and the explore sweep, one walk, one process:

```
SHOWDOWN_PATH=... GAMES=800 EVERY=2 N_LIST=40 EXPLORE_LIST=0,0.5,1.0 \
  DUMP=rollout-r1-E1-rows.jsonl \
  node --max-old-space-size=4096 engine/rollout_r1.js

node engine/rollout_r1_artifact.js data/rollout-r1-E1-rows.jsonl
```

`DUMP=` **resolves under** `data/`, **and a name already on disk is overwritten.** Use a release-stamped filename; the 2026-08-04 sweep nearly destroyed the only evidence for the incumbent arm this way.

Step 3 — R3:

```
SHOWDOWN_PATH=... GAMES=600 EVERY=3 N=600 TOPK=3 EXPLORE=1.0 \
node --max-old-space-size=4096 engine/rollout_r3.js
```

Step 4 — R7, the timing distribution on the named release (one process, 12 games):

```
SHOWDOWN_PATH=... MILTANK_TIMING=$PWD/data/.miltank-timing/r7.jsonl \
node --max-old-space-size=1536 engine/mew.js --n 12 --conc 1 \
--policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \
--seed 90001 --out data/.miltank-timing/r7-games.jsonl

node engine/miltank.js --reduce data/.miltank-timing/r7.jsonl \
--horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json
```

Read `search.leaf_calls` and `build.release_status` in that artifact before reading a single timing figure. They must be non-zero and `ON_RELEASE`.

Steps 5–7 — R5, then R6c, then R4 are already specified above and below in this file, with their shard commands. Each carries the same two preconditions: `release_status: ON_RELEASE` in the stamp, and a non-zero leaf-call count. Read every SPRT at the bound, once, on the cat of its shards.

And a caution the parity does not cover. `docs/SEARCH.md` item 5b records that the board's weather string has never meant anything to MEDICHAM. Until that is fixed, **a re-run of R1/R3/R4 would still be measuring a leaf that is blind to 60.8% of boards' weather.** Sequencing the two matters: fixing 5b after re-running the gates buys a second round of invalidation.

Read R4 correctly

R4 measured `--miltank-n 30`, uniform-random playout opponents, preview search disabled. It is a **floor, not a description**. It does not say "the bot is good".

What it does say is the thing worth keeping: the pre-fix baseline on the broken engine was -0.28 , and the same search with the same flags came back positive once the model was fixed. **A search is worth exactly what its model is worth.** That is why this division's open items are mostly about what the search *believes*, not how deep it goes.

And note the `PRE-CHANGE` markers in the generated block: those runs predate the current engine source. Under the frozen-release rule in [DIVISIONS.md](#) that is a re-run, not a judgement call.

The `--rollout-explore` default was re-earned, 2026-08-04

MEASURE retracted R1 that morning: the published 68.18% had no artifact, and the only committed row dump held the **explore=0** arm, on which R1 is UNDECIDED. `--rollout-explore` defaults to 1.0 and two comments cite that retracted figure as the reason.

It was re-run at explore=1 and it reproduces. Artifacts: `data/rollout-r1-explore-sweep.json` (the arm-vs-arm verdict, written by `engine/rollout_explore_sweep.js`) and `data/rollout-r1-explore1.json` (the gate, written by the existing `engine/rollout_r1_artifact.js` from `data/rollout-r1-explore1-rows.jsonl`).

	explore=0	explore=0.5	explore=1.0	material
accuracy, horizon 20 (9,201 positions)	65.72%	67.58%	67.97%	65.27%
accuracy, horizon 60 (4,487 positions)	64.21%	66.50%	67.46%	63.78%
ECE	0.196	—	0.104	0.050

	explore=0	explore=0.5	explore=1.0	material
share saturated in the 0–10 / 90–100 bin	50.7%	—	29.4%	—

Paired, on the identical sample: **+2.25 points, 95% CI [1.31, 3.19]**, monotone in explore at both horizons. The published 68.18% lands at 67.97% and the published +2.91 over material lands at +2.71 [1.60, 3.82] — the retraction was right about the *provenance* and the claim survives it.

Three things the re-run settled that were not the question:

- **The committed greedy dump was NOT clobbered.** `DUMP=` resolves under `data/`, so the command `MEASURE` left would have overwritten the only evidence for the incumbent arm. New filename used.
- **The "64.42% for greedy" half does not reproduce** — greedy measures 65.7% on both the committed dump and a fresh run on the current engine. Same sign, gap 2.25 not 3.76. The two comments overstate it and should be restated against the artifact.
- **Unfinished payouts are not the mechanism.** `battleResult` does score bodies-then-HP whether or not the battle ended, but 99.5–99.8% of payouts end by an actual wipeout at every explore setting and at both horizons. Cap-hits are 0.2–0.5%. Exploration makes payouts *longer* (4.4 → 6.1 mean turns), not truncated.

This is a verdict on a JUDGE, not on a player. It does not say `explore=1.0` wins more games, and `engine/mew.js` exposes no `--miltank-explore`, so the A/B that would say is not currently runnable. R4 was itself run at 1.0 and cannot arbitrate its own setting.

The live budget — SETTLED 2026-08-04 from the Showdown source, and it was not 45 seconds

This section previously said "a 7-minute chess clock with a 45-second cap on any one decision", flagged as an **unverified rules assumption** (PRIORITIES #39). It has now been read out of the implementation we actually play against, and **two of its three numbers were wrong**. Correcting the diagnosis, not just the number:

```
config/formats.ts gives [Gen 9 Champions] VGC 2026 Reg M-B the ruleset ['Flat Rules', 'VGC Timer',
'Open Team Sheets']. data/rulesets.ts vgctimer is, verbatim:

Timer Starting = 420      Timer Grace = 90      Timer Add Per Turn = 0
Timer Max Per Turn = 55   Timer Max First Turn = 90
Timeout Auto Choose      DC Timer Bank
```

and `server/room-battle.ts` says what those do:

fact	line	consequence
<code>secondsLeft = starting + grace</code>	:210	the bank is 510 s , not 420
<code>turnSecondsLeft = Math.min(secondsLeft, maxTurnTime)</code>	:327	the per-turn cap is DRAWN from the bank. One clock, not two
<code>addPerTurn = 0</code> , and <code>updateTurn</code> adds it	:306	no refill. A true bank, exactly as feared
<code>maxPerTurn = 55</code>	ruleset	the per-decision wall is 55 s , not 45
<code>maxFirstTurn = 90</code>	:326	team preview gets 90 s , from the same bank
turn expires, bank alive → <code>>{slot} default</code>	:451-453	a server-chosen move , not a loss
bank hits 0 → <code>forfeitPlayer(..., 'lost due to inactivity.')</code>	:455	you lose the game

Three further facts read on 2026-08-04, two of which correct the table above

The section above was right that the clock is one drawn bank and wrong about its size. Corrected against the source rather than argued:

1. THE BANK IS 420 s, NOT 510. The grace is use-it-or-lose-it and is not bankable. :209 does initialise `secondsLeft = starting + grace = 510`. But `updateTurn` at :305-306 runs

```
player.secondsLeft = Math.min(player.secondsLeft + addPerTurn, this.settings.starting);
```

on **every new turn**, `addPerTurn` is 0 and `starting` is **420**. So `Math.min(510, 420)` fires on the second timed request and the 90 s of grace is *clamped away*. It is spendable only on the first timed request.

`updateTurn` returns early the very first time (:270-274 , `this.turn === null`), which is exactly why the clamp lands on the second request and not the first.

Consequence: `budgetMs: 20000` **buys 21 decisions, not 24**. The figure this file replaced — "21 decisions" — was right by accident, off a wrong premise; the corrected derivation is $420,000 / 20,000 = 21$, with preview free because it comes out of the grace that is about to be clamped anyway.

2. THE BANK DOES NOT TICK WHILE THE TIMER IS OFF, so the mid-game switch-on is benign. `secondsLeft` is decremented only inside `nextTick` (:353), which is scheduled only by `nextRequest` (:341), which returns at :320 when `!this.timerRequesters.size . start()` then calls `nextRequest` itself (:239-240). So an opponent typing `/timer on` at turn 9 **does not find a bank MILTANK has already spent** — it finds a full one, and the 90 s `maxFirstTurn` allowance is granted on whatever turn the timer starts, because `isFirstRequest` (:167 , :326) is still true. The premise that the constraint arrives with unknown consumption already charged is **false**, and that makes the design easier rather than harder.

3. THE UNIT THAT SPENDS THE BANK IS A REQUEST, NOT A TURN. A post-KO replacement is its own request with its own 55 s window off the same bank — `updateTurn` returns at :286 for a mid-turn request without clamping and without adding. So a game with KOs has more requests than turns, and "expected remaining decisions" must be estimated in **requests**. Charging is also quantised: `TICK_TIME = 5` (:41) and each tick subtracts five whole seconds, so 12 s of thinking costs 15 s.

What MILTANK can and cannot observe about the clock

It can observe the bank exactly, and that was the surprise. `room-battle.ts:332` sends the player, privately, on **every request while the timer is on**:

```
|inactive|Time left: 55 sec this turn | 420 sec total | 90 sec grace
```

so `turnSecondsLeft` and `secondsLeft` are both **observed, not inferred**. Two parsing traps: the `total` field is `secondsLeft - grace` (:330-332), so the true bank is `total + grace` and reading `total` alone under-reads by 90 s; and the grace field is simply absent once it is gone. `|inactive|Battle timer is ON:` (:237) and `|inactiveoff|Battle timer is now OFF.` (:257) bracket the on/off state, and both are room-level so both players see them.

But nothing reads any of it. `engine/mag_bot.js` handles **zero** `|inactive|` lines — grepped, not assumed. So the capability exists in the protocol and is absent from the bot, which is CLAUDE.md's signature failure shape verbatim. `engine/miltank.js` now exposes `bot.noteClock(line|{turnSec, totalSec, graceSec})` and counts its calls in `bot.clockStats().notes` ; **that counter is 0 in every run to date and will stay 0 until OPS wires the handler in `mag_bot.js` , which is not SEARCH's file.**

Therefore the adaptive rule is designed against the worst case and does not depend on the observation. With no observation it assumes the timer has been on since the first request and charges itself its own tick-rounded wall clock from a full 420 s. If the timer is actually off it has throttled for nothing — a weaker search, not a lost game. If the timer comes on at turn 9 the estimate has over-charged for turns that were free, so it under-spends. Both errors land on the safe side, which is the only property that function is allowed to have.

The shipped numbers against the walls

	shipped	the wall
in-game decision	budgetMs 20,000	55,000 ms , capped by whatever is left in the bank
team preview	previewMs 15,000	90,000 ms , and it comes out of the grace, so it is free
the whole game	21 decisions at 20 s	420,000 ms, no refill
post-KO replacement	no budget at all until 2026-08-04	its own 55 s request off the same bank

Per decision MILTANK is nowhere near the wall. The binding constraint on one turn is budgetMs , which we chose. **Per game it is genuinely tight**, and it is invisible in every H2H we run because mew.js has no clock and both arms are equally free — the same *testing environment ≠ playing environment* error CLAUDE.md names, one layer out (PRIORITIES #39a).

R6 — the per-decision wall-clock distribution, MEASURED 2026-08-04

Artifact: data/miltank-timing-r6.json , rows in data/.miltank-timing/r6.jsonl . 12 self-play games, --policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 --seed 90001 , **120 decisions across 11 recorded games**, the 7,264-team clean pool (announced by the run, **not** --meta-teams), one process, Ryzen 7 7735HS / node v24.15.0.

THE BUILD THIS DESCRIBES, and it is already PRE-CHANGE. medicham2-browser.js b1b3ea94d5c3 , rollout_leaf.js 974c94d92398 , board.js abeb747f3219 , miltank.js 2cdf6f5b0924 . The run window was 19:17:21–19:25:29 UTC; ENGINE wrote data/abra-tags.js at 19:32:34 and medicham2-browser.js at 19:33:47, so **the tree moved seven minutes after the last row**. All 11 per-game stamps carry one digest, so the run is internally consistent — the digest is taken at module load, not at the first row, precisely so it cannot describe a file edited underneath a running process. **A duration is a fact about a machine under a load and about a build.** This one is not R2's mistake (PRIORITIES #14) because it says which; it is nonetheless a distribution for a build that no longer exists, and the weather-boundary fix makes playouts longer, so **treat every figure below as a LOWER BOUND on the post-fix engine.**

	median	p90	p99	max	over the 55 s turn wall
all decisions (n=120)	2,169 ms	9,977 ms	23,812 ms	23,866 ms	o
in-game move (n=98)	2,234 ms	13,891 ms	23,866 ms	23,866 ms	o
team preview (n=11)	2,076 ms	4,041 ms	4,195 ms	4,195 ms	o — but censored, see below
post-KO replacement (n=11)	445 ms	607 ms	4,528 ms	4,528 ms	o

A single decision cannot breach the 55 s wall at shipped settings. The worst of 120 was 23.9 s against a 55 s cap, on menus running 2–49 joint options. The brief's worse case — one decision exceeding the per-turn wall — **does not occur**, and that is the good half of this result.

budgetMs **IS A CHECKPOINT, NOT A DEADLINE, and that is a real finding.** 9 of 98 move decisions finished over the configured 20,000 ms, by up to **3,866 ms**. The budget is tested between finalists (miltank.js , the finalists loop), so whichever finalist is in flight runs to completion past it. The effective

per-decision cap is `budgetMs + one finalist evaluation` ≈ 24 s at $n=200$. It is comfortably inside 55 s today; it is not a bound anyone should quote as one.

Per game, against the 420 s bank:

	p50	p90	max	over the bank
requests per game	10	13	14	—
total spend per game	25.2 s	128.5 s	137.1 s	0 of 11

The worst game observed spent 33% of the bank. To forfeit, a game would have to cost roughly three times the worst one measured.

How long a real game actually is — 30,396 non-forfeit ladder games

`node engine/miltank.js --horizon data/games.ladder.jsonl`, folded into the artifact. Counted in **requests** (turns plus turns in which one of our bodies fainted, because a replacement is its own request off the same bank):

	p50	p75	p90	p95	p99	max
requests per game	9	11	13	15	19	74

0.58% of games exceed 21 requests, 0.30% exceed 24, 0.08% exceed 34. And the self-play run's own request counts (p50 10, p90 13) reproduce the store's (p50 9, p90 13) almost exactly, so the harness is a fair — mildly conservative — proxy for game length.

The verdict, stated plainly because it is not the one this was expected to produce

The shipped flat `budgetMs: 20000` is not currently a forfeit risk, and the measurement says so rather than the argument. Two independent legs:

- *Observed:* 0 of 11 games came within $3\times$ of the bank; worst was 137 s of 420 s.
- *Worst case:* even charging the full 20 s to every request, only **0.58%** of real games have enough requests to empty the bank, and MILTANK actually spends a median of 2.2 s.

So the adaptive rule is **a guard against a 0.6% tail and against an engine that is about to get slower**, not a fix for a live bleeding problem. That is a weaker case than PRIORITIES #39a assumed and it should be said out loud before anyone spends 420 games on R6a. **What has NOT been retired is the environment mismatch itself** — nobody has ever watched MILTANK play with a clock running, and the two facts that make the flat constant survivable (games are short, the search is usually fast) are properties of *this* build on *this* pool.

Preview is censored here and is the one number this run cannot give you

`mew.js` hardcodes `previewMs: 4000` and SEARCH may not edit it. **One of 11 previews truncated at 16 of 90 brings in 4,195 ms**; the other ten completed 90/90 in 2.1–3.2 s. Extrapolating the censored one at its own ~ 262 ms per bring gives **~ 23.6 s for a full preview** — which would truncate against the shipped `previewMs: 15000` too, and is still far inside the **90 s** first-turn wall. So preview cost varies $\sim 9\times$ with the team, and **preview is `previewMs` -bound, not wall-bound.**

Given that preview time comes out of grace that `updateTurn` is about to clamp away regardless, `previewMs: 15000` **is spending 17% of a free 90 s allowance.** Raising it is an accuracy decision, not a clock one, and it belongs with the preview-calibration item (#38) rather than here.

Adaptive spend — implemented 2026-08-04, behind a flag, DEFAULT OFF

`engine/miltank.js` now carries the rule this file asked for. `clock: false` is the default and the OFF path is byte-for-byte the player R4 measured, so nothing changes in a live game without a deliberate decision.

The rule, in one line: `spend (bank - reserve) / expected remaining requests`, clamped under the per-turn wall with a safety margin, floored so a starved tail still searches something. Four properties it was given on purpose:

- **It can only ever lower the budget, never raise it.** `budgetFor()` takes `min(adaptive, configured)`. A rule that could also spend *more* would need accuracy evidence of its own; this one needs only to not be worse.
- **The reserve is asymmetric, because the failures are.** Overrunning the *turn* costs one server-chosen move (`Timeout Auto Choose`, :451-453); emptying the *bank* forfeits the game (:455). So the reserve sits on the bank (`clockReserveMs`, 45,000 — nine ticks) and the turn gets only a tick-quantisation margin (`clockSafetyMs`, 10,000 under the 55 s wall).
- **The horizon is a high quantile of requests per game, not the mean** (`clockHorizonRequests`, default **19** = the measured p99 above) — the same reason the distribution is being measured at all. It is also floored by `clockTailMin` (8), because past the horizon `EXPECT - requests` collapses to 1 and the throttle silently switches *itself off* exactly when the game has proved it is a long one. Caught by driving the clock through 40 requests, not by reading it.
- **It bounds the post-KO replacement search, which had no deadline whatsoever.** Five candidates at `2 × ROLLOUT_N` on a request that draws its own 55 s from the bank was the one decision nothing capped. Only enforced when the flag is on.

knob	default	meaning
<code>clock</code> / <code>MILTANK_CLOCK=1</code>	off	adaptive spend against the bank
<code>clockReserveMs</code>	45,000	bank held back; a bank timeout is a forfeit
<code>clockMinMs</code>	1,500	floor, so a starved tail still searches
<code>clockSafetyMs</code>	10,000	margin under the 55 s per-turn wall
<code>clockHorizonRequests</code>	19	planning horizon in requests — the p99 measured over 30,396 ladder games
<code>clockTailMin</code>	8	never plan for fewer than this many more requests, so the throttle survives past the horizon
<code>clockEarlyDefer</code> / <code>MILTANK_EARLY_DEFER=1</code>	off	stop before the finalist round on positions heading for the tie band
<code>timing</code> / <code>MILTANK_TIMING=<path></code>	off	write the per-decision wall-clock artifact

`clockEarlyDefer` is not a timing lever and must not be argued as one. MILTANK hands turns back to MAG *after* paying for the finalist round. **Measured in R6: 31.6% of move decisions deferred, and they consumed 30.5% of the total spend** — deferred and chosen decisions cost almost identically (4,639 ms against 4,585 ms mean), so the saving really is proportional and **30.5% is the ceiling on this lever**, not the "roughly a quarter" this file previously guessed. The screen already has an opinion about the spread; it is only noisier. The estimator subtracts the expected pure-dice range of K estimates of one true value ($\approx 2.8\sigma$ at the screen's `n`) from the observed screen spread and asks whether what is left clears the **final** round's tie band. **It changes what gets clicked on every turn where the screen and the finals would have disagreed, so it gets its own SPRT arm.** Its bias is stated rather than hidden: it is biased *low*, which makes it defer more often than it should.

Why an environment variable rather than a flag. The A/B has to run through `engine/mew.js`, which is MEASURE's file, and the live path is `engine/mag_bot.js`, which is OPS's. SEARCH cannot add `--miltank-clock` to either — the same one-liner PRIORITIES #33 already owes `--miltank-explore`. The env var is recorded in every timing row, so a result is still attributable to the lever. Replace it with a real flag when #33 lands; do not leave two ways to set one thing.

R6 — the validation. SPEC. NOT RUN.

It is three questions and only one of them is an H2H. Collapsing them is the mistake this spec exists to prevent.

R6a — the DIVERGENCE screen first, and it is probably where this stops

Do not open with the 420-game H2H. It is a null by construction and the probe says so before any games are played. Driving the clock through 40 requests at the horizon and reserve that ship:

what MILTANK wants per request	requests where adaptive ≠ flat	first divergence
4.6 s — the R6 measured mean	0 of 40	never
20 s — every request at the full budget	29 of 40	request 12

At the spend actually measured, **the throttle never engages at all:** $(420 - 45)/19 \approx 19.7$ s is already above what MILTANK asks for, and `budgetFor` only ever takes the smaller of the two. That is the design working — it is a guard, and a guard that fires when nothing is wrong is a bug — but it means an H2H between the two arms would compare **two identical players on ~99.4% of games** and return a null that says nothing about the rule.

So R6a is a divergence count, not an SPRT. Run ~40 instrumented games with `MILTANK_TIMING` set on both arms and count the decisions whose `budget` differs from `budgetMs`. Publish that count.

- **If divergence is ~0** — the expected outcome — the rule is inert in normal play, the flag stays off, and **there is nothing to SPRT.** That is the honest end of this item and it costs 40 games, not 420.
- **Only if divergence is material** does the H2H below become worth its cost.

R6a' — the H2H, conditional on R6a showing divergence (~420 decisive pairs)

Paired and seed-matched exactly as R4. **Arm 1 is the challenger:** `MILTANK_CLOCK=1`. Arm 2 is the shipped flat `budgetMs: 20000`. Everything else identical — `--miltank-n 200`, `explore 1.0`, `foe uniform`, turns 60, same seeds, same team pool, pool announcement recorded. `mew.js` having no clock is *correct* here: it isolates the search-quality cost of throttling from the forfeit benefit, which is R6b's job.

Read once, at the bound. `node engine/sprt.js <cat of shards>`.

One behaviour the H2H must be told to expect, or it will be read as a bug. When the bank falls under `clockSafetyMs + 2 ticks` the budget goes to zero, the screen cuts every pair, and MILTANK falls back to MAG for the rest of the game. That is deliberate — with 10 s of bank left, an instant imitation move is strictly better than a forfeit — and it will show up as a burst of fallbacks at the end of long games in the challenger arm only.

R6b — how often does the flat constant actually forfeit? (NOT an H2H) — ANSWERED

An H2H in `mew.js` structurally cannot see this, because mew has no clock, so both arms are free and the forfeit never happens in either. It is answered from the timing artifact instead, and it has been: `games_over_bank_pct` is **0.0** (0 of 11; worst game 137 s of 420 s), and only **0.58%** of 30,396 real games have enough requests to empty the bank even at the full 20 s every time. **This is the number that decides the item and it says the lever is a guard, not a fix.**

It must be re-read after the engine release, because the weather-boundary fix lengthens playouts and every figure here is a lower bound on the post-fix build. `node engine/miltank.js --reduce <rows> --horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json .`

R6c — `clockEarlyDefer` (a separate H2H, never confounded with R6a')

Arm 1 `MILTANK_CLOCK=1 MILTANK_EARLY_DEFER=1`, arm 2 `MILTANK_CLOCK=1`. Same size, same pairing. Running it inside R6a' would make a play result unattributable between two levers, which is the failure "levers are per arm" exists to stop.

This is the one of the three worth running even though the clock does not bind, and the reason is not the clock at all: it buys **30.5% of the search budget back** on positions the search then throws away, and that time can be spent on `--rollout-n` instead. Its risk is bounded and stated — it can only change decisions that the finalist round would have deferred anyway or that the screen misranks. Size it at ~420 decisive pairs; the null it has to beat is "no worse".

The rule for all three

Run them after the Po.5 release boundary. Started before, they are born `PRE-CHANGE` and describe a build that stopped existing — already true of every R4 shard on disk. And every run must stamp `n_measured / n_unit`, the engine source digests, node version and machine: a duration is a fact about a machine under a load, which is precisely why PRIORITIES #14 says R2 is re-run or nothing. Do not let R6 become a second R2.

The commands. PREPARED, NOT RUN.

Four processes, not six — RAM is the ceiling and `FreePhysicalMemory` was 2.33 GB when R6 ran, which is **one** process. Check it before choosing a number. `SHOWDOWN_PATH` is required.

R7 — re-measure the distribution on the named release (one process, ~10 min, 12 games):

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
MILTANK_TIMING=$PWD/data/.miltank-timing/r7.jsonl \
node --max-old-space-size=1536 engine/mew.js --n 12 --conc 1 \
  --policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \
  --seed 90001 --out data/.miltank-timing/r7-games.jsonl

node engine/miltank.js --reduce data/.miltank-timing/r7.jsonl \
  --horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json
```

R6a — the divergence screen (same command, `MILTANK_CLOCK=1` added, shard 0..3 by `--seed`):

```
SHOWDOWN_PATH=... MILTANK_CLOCK=1 MILTANK_TIMING=$PWD/data/.miltank-timing/r6a-s<N>.jsonl \
node --max-old-space-size=1536 engine/mew.js --n 10 --conc 1 \
  --policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \
  --seed 9100<N> --out data/.miltank-timing/r6a-s<N>-games.jsonl
```

Cat the shards, then `--reduce` once, and count rows where `budget < budgetMs`. **Read it at the bound, once.**

R6c — the `clockEarlyDefer` H2H, only after R7 has re-stamped the leaf. Arm 1 is the challenger and carries `MILTANK_EARLY_DEFER=1`; both arms carry `MILTANK_CLOCK=1`. ~420 decisive pairs, sharded four ways, then `node engine/sprrt.js <cat of shards>` **once**.

Open

6. `mag_bot.js` PARSES NO `|inactive|` LINE — FILED FOR OPS, NOT FIXED HERE

The server hands the player its exact bank on every request while the timer is on (`room-battle.ts:332`) and **the bot throws it away**: `grep engine/mag_bot.js` for `inactive` and it returns zero. `engine/miltank.js` now exposes `bot.noteClock(line)` and counts calls in `bot.clockStats().notes` ; **that counter is 0 and stays 0 until something calls it**. Under CLAUDE.md the capability is therefore assumed broken.

It is one handler in the socket loop, and `mag_bot.js` is **OPS's file and the live path** — SEARCH must not touch it while Will may be playing. Two things whoever wires it must get right, both read out of the source rather than guessed:

- the bank is `total + grace` , not `total` . `:330-332` prints them apart and the grace field is absent once it is gone. Reading `total` alone under-reads the bank by 90 s at the start;
- `|inactive|Battle timer is ON: ( :237 )` and `|inactiveoff| ( :257 )` are **room-level**, so both players see them, but the `Time left: line at :332` is sent to **that player only** — do not try to read the opponent's clock off it.

Until then the adaptive rule runs on the worst-case self-charged estimate, which is deliberate and is strictly on the safe side, but it is an estimate.

o. ~~The in-game leaf and the preview leaf are different players~~ — CLOSED 2026-08-04

MEASURE's calibration (2026-08-04) ranked the **preview** leaf at 53.22% ($p<1e-4$) and the **in-game** leaf at 50.99% ($p=0.47$). Those were not two settings of one thing. `engine/miltank.js` held a **second, hand-rolled** **playout loop** — `battleInit` / `battleTurn` directly, deterministic greedy on both sides — that never called `rolloutWinProb` . MILTANK shipped two leaves and only one of them was ever swept.

There is now one playout. `rollout_leaf.runPlayout` is the single implementation; both leaf entry points call it, and the preview calls `rolloutWinProb` like everything else. What used to be a second implementation is now three parameters:

	preview before	preview now	why
playout policy	deterministic greedy, both sides	<code>explore</code> (default = the in-game 1.0)	one player, not two. <code>previewExplore: 0</code> restores greedy
opponent model	<code>chooseAction</code> , ignored -- <code>miltank-foe</code>	<code>foePolicy</code> — the flag now reaches preview	the foe A/B was silently not running at preview
game start	<code>battleInit({seeded:true})</code> on a game that had not started	<code>seeded:false</code>	see item 1 below
horizon	hardcoded 60	<code>turns</code>	it happened to agree; now it cannot drift
dice across brings	a fresh seed per bring	common random numbers , one seed for the whole preview	each bring was judged on its own independent draws, so the difference between two brings sat under the noise. Same fix, same reason, as <code>replSeed</code> in the post-KO search

This changes what the bot leads with, and the 53.22% no longer describes a policy that ships. That figure was measured on the greedy loop; it is now a number about a deleted implementation and must not be quoted for the shipped preview. Re-measuring it is MEASURE's, after the release boundary.

Two things this does NOT settle. The old 53.22%-vs-50.99% contrast was also measured on different position distributions (fresh full teams against mid-game boards), so it was confounded twice over and unifying removes only one of the confounds. And nothing has measured that preview *wants* `explore=1.0` — defaulting it there is a decision to have one player until something says otherwise.

ob. The preview enumerated brings it could not field — fixed the same pass

Caught by the smoke run for the unification, not by anything that was watching. The bring enumerator mixed **positions in the buildable list** with **team indices**: the lead pair came from one and the back pair from the other. They coincide when all six Pokemon build, which is why it survived. On a team with two unbuildable bodies, measured:

- 19 brings enumerated where exactly **6** exist
- **18 of the 19** named a Pokemon the search had just printed as unbuildable
- 15 "distinct" brings out of a true set of 6

and the missing body was then silently dropped by `.filter(Boolean)`, so a **three**-Pokemon bring was scored and reported as a four-Pokemon one. Full six-buildable teams enumerate 90 before and after, so this only ever bit the case the "drop what cannot be built" path exists for.

1. The preview seeded a game that had not started — fixed 2026-08-04

`chooseTeamPreview` called `battleInit(..., {seeded: true})`. `seeded` exists to stop a **mid-battle** leaf re-firing entry effects that already happened — re-running Intimidate would drop the same Attack a second time on every board with an Incineroar. At team preview nobody has entered yet, so it suppressed the entry effects entirely, and those are most of what a lead decision is. Deciding a lead is largely deciding who eats an Intimidate, and the search could not see one.

Measured directly against the engine, Torkoal + Incineroar leading into Garchomp + Gholdengo:

```
seeded=true    weather=null    weatherT=0    foe atk stages=[0, 0]
seeded=false   weather="sun"    weatherT=5    foe atk stages=[-1, -1]
```

So before the fix every preview playout ran with **no Drought, no Drizzle, no Sand Stream, no Snow Warning, no terrain setter and no Intimidate on turn one** — the whole switch-in-ability class, deleted, in the one decision they matter most for. Fixed by making `seeded` a parameter of `rolloutWinProb` (default unchanged at `true`) and passing `false` from preview only.

1b. Opponent model — the A/B in flight

Playouts move uniformly at random. Real Charizard clicks Protect 60.6% of the time, not 25%. `--miltank-foe prior` exists and is being compared against uniform; shards land in `data/.mew-shards/foe-s*.jsonl`.

If prior wins this changes every evaluation in the project, because every leaf number was computed against a foe that does not exist.

Counter-consideration, and it is not small: a fully random rollout has repeatedly judged *better* than a greedy one. Do not assume a more realistic playout is a better estimator — that is exactly what the A/B is for. Read it at the bound.

2. Which mega to take

Currently "the lead keeps it", which is arbitrary. It should be a search decision, and it is cheap to make one — only two-stone brings branch at all.

"Biggest stat gain" was **measured and discarded**: every Champions mega is +101 to +104. Do not re-propose it.

3. Team quality

`--meta-teams` yields 169 teams, but the base filter is **completeness, not quality** — so the pool contains Mickey Mouse teams: real, open-sheet, and still terrible. The pool is announced on every start, on or off. Read the announcement before attributing a result to a lever.

4. Leaf calibration blocks everything here

Every decision this division makes is an argmax over the leaf. If the leaf is uncalibrated, a better search is a better-aimed error. This is MEASURE's item, not SEARCH's — but SEARCH should know that a null result here may not be about the search at all.

5. applyMegaWeather — FIXED 2026-08-04, with #40b, and measured

engine/rollout_leaf.js called applyMegaWeather(S) and then assigned the caller's field over the top — S.field.weather = f.weather || '' — unconditionally, one line later. battleInit({seeded:true}) leaves S.field.weather null, so the guard if (S.field.weather) return never fired, the function always ran, and its write was always discarded. Mega Charizard Y stood in clear weather in every mid-battle rollout this project ever ran.

Fixed by ORDER: the caller's field is applied first and applyMegaWeather second, so the guard arbitrates instead of being overwritten. A real weather the board reports still wins.

Two corrections to #40b as filed, both measured rather than reasoned.

- The raw ability read was not returning a wrong answer on this path. dmgMon already calls effAbility itself, so a Charizard + Charizardite Y body arrives at the function carrying drought, not blaze. Probed directly (species=charizard item=Charizardite Y → body ability drought; same for Tyranitar→sandstream, Abomasnow→snowwarning). #40b is a latent hazard here — live the moment a caller omits dex — not the live defect the filing describes. #37 alone was the visible bug. Routed through effective() anyway, and rollout_leaf.js now holds 0 raw reads of a transforming field against a baseline of 0, down from 2 at bd8f388.
- The field the fix actually needed was mega, not the ability. The discarded version had no mega check at all — it took the weather of the first ACTIVE with a weather-setting ability, mega or not. Landing #37 without adding that gate would have invented weather: a Torkoal standing on a board the tracker says is clear is a legal observed state, and re-setting its sun overrules the one thing that knows. The board is authoritative for a body that is what the board says it is; a mega is the sole exception, because dmgMon has already upgraded it to a forme the real game has not seen. So #37 and #40b were coupled, but not for the reason filed.

Parity, same boards, same seeds, both entry points, HEAD leaf against the working tree, in one process so the engine cannot differ between arms.

	before	after
boards walked	60	250
rolloutWinProb different	0	14
rolloutAfterActions different	0	14
boards moved on at least one	0	15
boards moved that are NOT a mega-setter-on-a-clear-board	0	0

15 of the 16 boards that hold a mega weather setter with no board weather moved; the 16th sits at 0.975/1.000 and is saturated at n=40. Nothing else moved at all — the reorder's only other side effect, weatherT no longer being left at 5 on boards that already have weather, is inert, and the parity proves it rather than assuming it.

Effect size, and it is not small: mean |Δ| 9.67 pt on rolloutWinProb (max 17.5) and 18.33 pt on rolloutAfterActions (max 62.5, n=24 and correspondingly noisier). The direction is the correctness evidence — Charizard-Mega-Y's sun is worth +11.0 pt to the side that owns it and -12.5 pt to the side that faces it. Tyranitar-Mega's sand moves ±5 pt, which is the right order for a weather whose main modelled effect is a Rock special-defence multiplier.

Direct counter, because a parity delta is indirect. `battleTurn` wrapped, field read on turn 1 of every playout, 250 boards × 40:

```
HEAD leaf      : 0 of 9,040 playouts began in a weather MEDICHAM can read  (0.00%)
working tree    : 640 of 9,040                                           (7.08%)   sun 480, sand 160
```

Rate in the corpus sample: 32 of 250 boards (12.8%) hold a mega weather setter among the actives — Charizard-Mega-Y 17, Tyranitar-Mega 15 — and 16 of 250 (6.4%) are the ones with no board weather that #37 can move. **That is the honest exposure figure, and it is not the "~26% of format usage" the filing quoted:** 26% is megas in general, most of which set no weather.

Leaf cost: 17.53 → 16.93 ms per call at n=40 over 120 boards, i.e. no regression (one machine, one load — an order of magnitude, not a figure).

The unseeded preview path is untouched, and that was checked rather than argued: 12 preview-shaped calls (`seeded:false`, `buildTeams`, a mega setter on both sides) are identical across the two leaves.

The engine moved under this work. `engine/medicham2-browser.js` was committed and then modified again between the before-run and the after-run — 4 of the first 60 HEAD-leaf values differ across the two runs. The parity verdict survives it because both arms run in one process against one engine, and the after-run carries its own before-state column measured on the current engine. The absolute numbers above describe the tree at `9a4f82d` plus uncommitted ENGINE work, not a named release.

5c. THE SEARCH USED A THIRD TERRAIN VOCABULARY — FIXED 2026-08-04, and the effect is near zero

This is 5b's sibling, one layer further out, and it is the reason ENGINE's terrain fix measured nothing. ENGINE routed every terrain read in `medicham2-browser.js` through `terrainId()` and then counted that **0 of 863 terrain-carrying boards reach the leaf at all**. The defect was not in the engine: `engine/miltank.js:794` and `engine/rollout_r1.js:175` built the field object with

```
terrain: ['electric', 'grassy', 'misty', 'psychic'].find(t => board.hasField(t)) || ''
```

which are the **engine's** words probed against a Board that stores the dex's `electricterrain`, `grassyterrain`, `mistyterrain`, `psychicterrain`. **Three vocabularies, and the one doing the asking was the one nothing spoke.** Reproduced on an independent walk before the fix: the short-word probe matched **0 of 3,256** boards.

Fixed with `rollout_leaf.terrainOnBoard(board)` — **one implementation, no fourth map.** It probes the board's own four keys and translates with `MEDI.terrainId`, exactly as `applyField` already translates weather with `MEDI.weatherId`. Three call sites now use it, and the third was a hole nobody had filed: `miltank.js`'s **post-KO replacement search had** `terrain: ''` **hardcoded**, so every replacement was judged on a bare field. `applyField` also now runs `f.terrain` through `terrainId`, because that boundary is handed both vocabularies and `terrainId` is idempotent.

Two deliberate choices, both recorded so they are not re-litigated:

- **Probe four named keys rather than walking** `board.pseudoWeather`. Trick Room lives in the same namespace, so a walk would hand `trickroom` to `terrainId` and score a bogus `MEDI.fails.terrainUnknown` on nearly every board. A swallowed-failure counter that fires when nothing is wrong is a counter that gets ignored.
- `engine/rollout_r3.js:98` **still passes** `terrain: ''` and is not SEARCH's file this pass. It is the same hole and it wants the same one-line call.

What it moved: NEAR ZERO, which is the expected honest answer

Both arms in one process against one engine, same boards, same seeds, so only the field object differs. 800 games, every 2nd board, n=40.

boards walked	3,256		
hits by the OLD short-word probe	0		
boards carrying a terrain	29 (0.891%; ENGINE's whole-corpus figure is 1.24%)		
which terrains	electric 19, psychic 10		
boards rolled out	205 — all 29 terrain boards, plus 176 terrain-free controls		
boards that moved	4		
controls that moved	0 of 176 — the control that says this is the terrain and nothing else		
mean \	Δ \	on the movers	8.75 pt (max 25.0)

All four movers are Psychic Terrain; not one of the 19 Electric Terrain boards moved. That is not a bug to hunt, it is the reader set: the engine consumes terrain in exactly four places — the Psychic Terrain priority block, Grassy Glide's priority, Hadron Engine (0 corpus uses), and terrainScaled (Expanding Force 182 uses, Rising Voltage 114). A 1.24% condition times a thin reader is a small number. 1.24% exposure and a 4/205 movement rate is the result; do not go looking for more.

One thing tripped over and FILED FOR ENGINE, not fixed here. The engine models no generic terrain type boost — Electric Terrain does not multiply a grounded Electric move, Grassy Terrain does not multiply Grass, Psychic Terrain does not multiply Psychic, and Grassy Terrain's end-of-turn heal and Misty Terrain's status block are absent. Probed directly: a Psychic-type move into Garchomp reads 103-123 with no terrain and 103-123 under psychic and under psychicterrain alike. terrainScaled covers only the two moves whose handler names a number. Recorded here rather than in docs/ENGINE.md because an ENGINE agent owns that file right now — the same reason 5b was recorded here — so it is not lost. It also bounds 5c: the movement above will grow when the generic boost lands, and only then.

5b. THE LEAF'S WEATHER STRING HAS NEVER MEANT ANYTHING TO THE ENGINE — FILED, NOT FIXED

Tripped over while measuring #37, and it is much larger than #37. applyField assigns f.weather straight into S.field.weather. f.weather comes from board.weather, which is Showdown's |-weather| line normalised, so its values are move names — sunnyday, raindance, sandstorm, snowscape. engine/medicham2-browser.js compares against sun / rain / sand / snow (:464, :486-487, :934). They have never matched.

Measured on the shipped engine, Charizard Flamethrower into Garchomp:

weather=""	61-72	weather="sun"	92-109	weather="sunnyday"	61-72
		weather="rain"	29-35	weather="raindance"	61-72

So the weather a mid-battle board reports is truthy enough to suppress a guard and meaningless to every formula. The turn-1 counter above says it exactly: 0 of 9,040 payouts on the HEAD leaf began in a weather MEDICHAM could read, while 5,320 of them had a weather string. 152 of 250 sampled boards (60.8%) carry one.

Not fixed here, deliberately. Correcting it moves roughly 65% of in-game leaf values — an order of magnitude more than #37 — and landing it in the same pass would have made the #37 parity unreadable, which is the exact confound #37 was deferred to avoid in the first place. It is an ENGINE item; SEARCH could not write `docs/ENGINE.md` this pass and it is recorded here so it is not lost. The translation point is `rollout_leaf.js applyField`, which now carries the finding in a comment.

It also bounds what #37 bought. With the board's weather inert-but-truthy, the mega's weather is applied only where the board reports *no* weather at all. Under a real weather the mega still stands in nothing — because the board's weather is nothing too. Fixing 5b will move #37's exposure up.

R5 — the action-ranking backtest. SPEC. NOT RUN.

This is the measurement nobody has done, and it is the one that decides whether the rollout leaf is worth its 142 ms. Every leaf number produced so far — R1, the explore sweep, the calibration — scores a **predictor of a game outcome from a fixed position**. A search leaf is asked something different: *which of ~63 candidate joint actions is best*. Those are not the same job. **Two leaves with identical Brier can order actions completely differently**, and ordering is the only thing a search consumes.

The question, and what is NOT being asked

*At a real decision point, does `rolloutWinProb` **order** the expressible joint actions differently from `materialP` on the post-action board?*

Not "which is more accurate" — that is R1 and it is answered. **Do not size this as a superiority test.** At R1's effect size a decision-level superiority test needs roughly **190,000 decisions**; this run is ~2,000 and would be catastrophically underpowered for that question. Agreement needs no effect size, which is the whole reason this is the affordable measurement.

Procedure

1. **Sample.** Walk clean corpus games with the same `JR.build` walker `engine/rollout_r2.js` and `rollout_r3.js` use. Take every 3rd board so one long game cannot dominate. Target **~2,000 decision points**. Record the game id on every row — the analysis clusters on it.
2. **Enumerate the menu the search actually sees.** `board.js` `candidates()` per slot, then the *same* expressibility filter `miltank.js` applies: drop anything MEDICHAM resolves as `pass`, and drop a pure-status click whose every effect is refused. Offering a cell the engine cannot express makes distinct options collapse into one and the agreement rate becomes an artefact. Corpus median is ~8 options a slot, so ~63 joint cells.
3. **Score each cell twice, on the SAME dice.** One seed per decision point, shared across all cells — common random numbers, the same variance reduction the post-KO search already uses. Without it the disagreement set is mostly noise.
4. **rollout:** `RL.rolloutAfterActions(board, side, {n: 200, explore: 1.0, foePolicy: 'uniform', maxTurns: 60, myClicks, seed, dex, field})`.
5. **material:** step **one turn** with the same forced clicks and the same seed, then evaluate the material estimator on the **post-action** board. It must be the post-action board: scored on the pre-action board, material returns the same number for every cell and has no ordering at all, which would produce a meaningless 100% disagreement. Because the stepped turn is stochastic, average it over **m = 8** steps under the same seed and report `m`.
6. **Use the material estimator R1 was scored against — the existing one, by name, wherever it lives.** Do not write a second one. FACTS ARE GLOBAL; two implementations of "count the bodies" will

disagree eventually and the disagreement will be invisible. Stamp which file it came from.

7. **Per decision point, record:** Spearman rho, Kendall tau-b (ties matter — a rollout at $n=200$ has granularity $1/200$ and material is coarser still), whether the two argmaxes are the same **cell**, whether the rollout's argmax is inside material's top 3, and the rollout argmax's rank under material.
8. **Across decision points, report WITH INTERVALS — not a winner.** Mean rho and mean tau with a 95% CI, and the argmax-agreement rate with a 95% CI. **Bootstrap clustered on GAME, not on decision.** Decisions inside one game are not independent and treating them as such is the single easiest way to publish an interval three times too narrow here.

The decision rule, stated before the run

- **If argmax agreement is ~90% or higher and the lower CI bound holds above ~85%**, the two leaves choose the same action almost always and the 142 ms is **provably wasted as an action ranker**. That is a real, publishable, negative result and it redirects the division onto the leaf itself.
- **If they disagree materially**, the **disagreement set is the sample the H2H should then run on** — those positions and only those are where the two players differ, which is a far cheaper and far sharper H2H than 420 random games.

Cost, and why ~2,000 is the size

	per leaf call	per decision (~63 cells)	2,000 decisions
$n=200$, explore=1.0, maxTurns=60	141.85 ms	8.94 s	~ 5.0 h , one process
$n=40$ screen	37.48 ms	2.36 s	~ 1.3 h , one process

Both leaf costs are R2's, and R2 is itself under re-run (PRIORITIES #14) — treat them as the order of magnitude, not as the figure. A 40-rollout screen followed by $n=200$ on the survivors is the cheap version, but note it changes the question: it measures agreement on the *shortlist*, not on the menu. Run the full version if it can be afforded.

Running it

Four processes, not six — ENGINE is live and PRIORITIES sets the working cap at 4. Shard by game id modulo 4, one JSON row per decision point:

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
  SHARD=<0..3> SHARDS=4 N_DECISIONS=500 ROLLOUT_N=200 EXPLORE=1.0 MAXTURNS=60 \
  node --max-old-space-size=4096 engine/rollout_r5.js --out data/.r5-shards/r5-s<0..3>.json1
```

Cat the shards, then reduce once. **Read it at the bound, once.**

What the run must stamp on itself, or it is not evidence

`n`, `explore`, `foePolicy`, `maxTurns`, `m`, the engine source digest, the material estimator's file and function, the corpus id and the team-pool announcement, node version and machine, and `n_measured` / `n_unit` — PRIORITIES #20 records R1 and R4 missing exactly those two fields; do not make it three.

Run it after the Po.5 release boundary. Started before, it is born `PRE-CHANGE` and describes a build that stopped existing — which is already true of every R4 shard on disk.

Running a comparison

Levers are **per arm**, and **arm 1 is the challenger** — check `winnerWeights` before ever "fixing" an analyser that looks broken. SPRT-gate it and read it at the bound, never during.

Size the run to the question: an H2H decides in roughly 420 games, not 200,000.

Done looks like

- A gated, artifact-backed SPRT verdict against a **named engine release**, not against HEAD.
- Every arm's flags recorded in the run, so a result can be attributed to a lever without guessing.
- The opponent-model A/B read once, at the bound, and written to an artifact.

Where this is going

`docs/MILTANK.md` §3.1 explains why the current best-response player is exploitable by construction. An opponent-aware playout is the first step toward an equilibrium player; ship it only if the A/B says so.